Date:10-10-2026

CO1-ASSESSMENT TOOL 1-APPLICATION BASED QUESTIONS

COURSE CODE: DSA0308

COURSE NAME: NATURAL LANGUAGE PROCESSING

REGISTRATION NO:192472193 **NAME:** C G CHARISHMA

Question 1: Resume Information Extraction System

1. Objective

To develop a Python program using Regular Expressions to extract candidate information such as name, email, mobile number, skills, experience, and profile summary from resumes. The program also checks candidate eligibility based on minimum experience and Python skills.

2. Procedure

1. Import the re module to use regular expressions.
2. Store multiple resumes containing candidate details in a list.
3. Use re.search() with suitable patterns to extract the name, email, mobile number, skills, experience, and summary.
4. Convert the extracted experience into an integer and check whether it is at least two years and whether the candidate has Python skills.
5. Display the extracted information and eligibility status for each candidate.

3. Python Code

```
import re


resumes = [
    """Name: Ananya Rao
```

```
Email: ananya@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL
Experience: 3 years
Summary: Python developer with machine learning knowledge.""",

    """Name: Ravi Kumar
Email: ravi@gmail.com
Mobile: 9876543211
Skills: Java, SQL
Experience: 1 year
Summary: Java developer.""",

    """Name: Meera Das
Email: meera@gmail.com
Mobile: 9876543212
Skills: Python, NLP, Machine Learning
Experience: 2 years
Summary: NLP developer with Python skills."""
]

for resume in resumes:
    name = re.search(r"Name:\s*(.+)", resume).group(1)
    email = re.search(r"Email:\s*(\S+)", resume).group(1)
    mobile = re.search(r"Mobile:\s*(\d+)", resume).group(1)
    skills = re.search(r"Skills:\s*(.+)", resume).group(1)
```

```
    experience = int(re.search(
        r"Experience:\s*(\d+)", resume
    ).group(1))
    summary = re.search(r"Summary:\s*(.+)", resume).group(1)

    eligible = experience >= 2 and re.search(
        r"\bPython\b", skills, re.IGNORECASE
    )

    print("Candidate Name:", name)
    print("Email:", email)
    print("Mobile:", mobile)
    print("Skills:", skills)
    print("Experience:", experience, "years")
    print("Profile Summary:", summary)
    print("Eligibility:", "Eligible" if eligible else "Not Eligible")
    print("-" * 40)
```

4. Input

The program receives three sample resumes containing the following details.

Resume 1

- Name: Ananya Rao
- Email: ananya@gmail.com
- Mobile: 9876543210
- Skills: Python, Java, SQL
- Experience: 3 years
- Summary: Python developer with machine learning knowledge.

Resume 2

- Name: Ravi Kumar
- Email: ravi@gmail.com
- Mobile: 9876543211
- Skills: Java, SQL
- Experience: 1 year
- Summary: Java developer.

Resume 3

- Name: Meera Das
- Email: meera@gmail.com
- Mobile: 9876543212
- Skills: Python, NLP, Machine Learning
- Experience: 2 years
- Summary: NLP developer with Python skills.

5. Output

Candidate Name: Ananya Rao

Email: ananya@gmail.com

Mobile: 9876543210

Skills: Python, Java, SQL

Experience: 3 years

Profile Summary: Python developer with machine learning knowledge.

Eligibility: Eligible

Candidate Name: Ravi Kumar

Email: ravi@gmail.com

Mobile: 9876543211

Skills: Java, SQL

Experience: 1 years

Profile Summary: Java developer.

Eligibility: Not Eligible

Candidate Name: Meera Das

Email: meera@gmail.com

Mobile: 9876543212

Skills: Python, NLP, Machine Learning

Experience: 2 years

Profile Summary: NLP developer with Python skills.

Eligibility: Eligible

Result — Step-by-step explanation

Step 1: Read the first resume.

The program takes Ananya Rao's resume and uses re.search() to find her name, email, mobile number, skills, experience, and profile summary.

For example:

- r"Name:\s*(.+)" extracts Ananya Rao after Name:.
- r"Email:\s*(\S+)" extracts ananya@gmail.com.
- r"Mobile:\s*(\d+)" extracts 9876543210.
- r"Experience:\s*(\d+)" extracts the number 3.

The \s* allows optional spaces, \S+ matches non-space characters, and \d+ matches one or more digits.

Step 2: Check Ananya's eligibility.

The program checks two conditions:

1. Her experience must be at least two years: 3 >= 2, which is true.

2. Her skills must contain the word Python. Her skills are Python, Java, SQL, so this condition is also true.

Since both conditions are satisfied, Ananya is marked Eligible.

Step 3: Process Ravi Kumar's resume.

The program extracts Ravi's details: ravi@gmail.com, 9876543211, Java, SQL, and one year of experience.

The experience condition 1 >= 2 is false. His skills also do not contain Python. Therefore, Ravi is marked Not Eligible.

Step 4: Process Meera Das's resume.

The program extracts Meera's details: meera@gmail.com, 9876543212, Python, NLP, Machine Learning, and two years of experience.

The experience condition 2 >= 2 is true, and her skills contain Python. Therefore, Meera is marked Eligible.

Step 5: Display the extracted details.

After processing each resume, the program prints all extracted information, displays the eligibility status, and prints a separator line using print("-" * 40).

Overall result: The resume information extraction system successfully extracts candidate details and identifies Ananya Rao and Meera Das as eligible candidates. Ravi Kumar is marked not eligible because he does not satisfy the experience and Python-skill requirements.

Question 2: Product Search System

1. Objective

To develop a Python product search system using Regular Expressions that supports exact, prefix, suffix, partial, and case-insensitive searches. The program displays matching products and the total number of matches.

2. Procedure

1. Import the re module and create a list of product names.
2. Define a function named search_products() that accepts a search pattern and search mode.
3. Use re.fullmatch() for exact searches, re.match() for prefix searches, and re.search() with $ for suffix searches.
4. Use re.search() for partial searches and re.IGNORECASE to make searches case-insensitive.
5. Display the matching products and calculate the total number of matches for each search.

3. Python Code

```
import re


products = [
    "Python Programming Book",
    "Python Data Science Course",
    "Java Programming Guide",
    "First Aid Kit",
    "Travel Kit",
    "Database Management System"
]
```

```
def search_products(pattern, mode):
    matches = []

    for product in products:
        if mode == "exact":
            matched = re.fullmatch(
                pattern, product, re.IGNORECASE
            )
        elif mode == "prefix":
            matched = re.match(
                pattern, product, re.IGNORECASE
            )
        elif mode == "suffix":
            matched = re.search(
                pattern + r"$", product, re.IGNORECASE
            )
        else:
            matched = re.search(
                pattern, product, re.IGNORECASE
            )

        if matched:
            matches.append(product)

    print(mode.capitalize(), "Search:", pattern)
```

```
    if matches:
        for product in matches:
            print(product)
    else:
        print("No matching products found")

    print("Total matches:", len(matches))
    print()

search_products("Python Programming Book", "exact")
search_products("Python", "prefix")
search_products("Kit", "suffix")
search_products("Data", "partial")
search_products("pYtHoN", "case-insensitive")
```

4. Input

The product list contains:

1. Python Programming Book
2. Python Data Science Course
3. Java Programming Guide
4. First Aid Kit
5. Travel Kit
6. Database Management System

The search operations performed are:

- Exact search: Python Programming Book
- Prefix search: Python

- Suffix search: Kit
- Partial search: Data
- Case-insensitive search: pYtHoN

5. Output

Exact Search: Python Programming Book

Python Programming Book

Total matches: 1

Prefix Search: Python

Python Programming Book

Python Data Science Course

Total matches: 2

Suffix Search: Kit

First Aid Kit

Travel Kit

Total matches: 2

Partial Search: Data

Python Data Science Course

Database Management System

Total matches: 2

Case-insensitive Search: pYtHoN

Python Programming Book

Python Data Science Course

Total matches: 2

Result — Step-by-step explanation

Step 1: Create the product list.

The program stores six product names in a list:

- Python Programming Book
- Python Data Science Course
- Java Programming Guide
- First Aid Kit
- Travel Kit
- Database Management System

The function search_products(pattern, mode) searches this list using the specified regular-expression pattern and search mode.

Step 2: Perform an exact search.

The program executes:

search_products("Python Programming Book", "exact")

The re.fullmatch() function checks whether the entire product name matches the given pattern. Only Python Programming Book matches exactly.

Therefore, the total number of matches is 1.

Step 3: Perform a prefix search.

The program executes:

search_products("Python", "prefix")

The re.match() function checks whether the pattern matches at the beginning of a product name.

The matching products are:

1. Python Programming Book
2. Python Data Science Course

Therefore, the total number of matches is 2.

Step 4: Perform a suffix search.

The program executes:

search_products("Kit", "suffix")

The program adds $ to the pattern, producing Kit$. The $ symbol indicates the end of the string.

The matching products are:

1. First Aid Kit
2. Travel Kit

Therefore, the total number of matches is 2.

Step 5: Perform a partial search.

The program executes:

search_products("Data", "partial")

The re.search() function checks whether the pattern appears anywhere within a product name.

The matching products are:

1. Python Data Science Course
2. Database Management System

The second product matches because the word Database begins with Data.

Therefore, the total number of matches is 2.

Step 6: Perform a case-insensitive search.

The program executes:

search_products("pYtHoN", "case-insensitive")

The re.IGNORECASE flag allows the pattern to match regardless of uppercase or lowercase letters.

Both Python products match:

1. Python Programming Book
2. Python Data Science Course

Therefore, the total number of matches is 2.

Step 7: Display the search results.

For each search, the program displays the search mode, pattern, matching product names, and total number of matches. If no product matches, it displays No matching products found.

Overall result: The product search system successfully performs exact, prefix, suffix, partial, and case-insensitive searches using Python regular expressions. It identifies the appropriate products and displays the correct match count for each search.

Question 3: University Registration Validation System

1. Objective

To develop a Python program using Regular Expressions to validate university registration details, including register number, institutional email, course code, semester, and mobile number. The program determines whether a registration is successful based on the validity of all fields.

2. Procedure

1. Import the re module and initialize the registration details.
2. Use re.fullmatch() to validate the register number against the required nine-digit format.
3. Validate the institutional email, course code, and mobile number using appropriate regular-expression patterns.
4. Check whether the semester is numeric and lies between 1 and 8.
5. Display the validity of every field and use all() to determine the final registration status.

3. Python Code

```
import re

register_no = "192472193"
email = "charishma@simats.ac.in"
course_code = "DSA03"
semester = "5"
mobile = "9876543210"

validations = {
    "Register Number": bool(
```

```
        re.fullmatch(r"\d{9}", register_no)
    ),
    "Institutional Email": bool(
        re.fullmatch(r"[\w.-]+@simats\.ac\.in", email)
    ),
    "Course Code": bool(
        re.fullmatch(r"[A-Za-z]{2,5}\d{2,4}", course_code)
    ),
    "Semester": semester.isdigit() and 1 <= int(semester) <= 8,
    "Mobile Number": bool(
        re.fullmatch(r"[6-9]\d{9}", mobile)
    )
}

for field, valid in validations.items():
    print(field + ":", "VALID" if valid else "INVALID")

if all(validations.values()):
    print("Registration Status: SUCCESSFUL")
else:
    print("Registration Status: UNSUCCESSFUL")
```

4. Input

The program receives the following registration details:

- Register Number: 192472193
- Institutional Email: charishma@simats.ac.in
- Course Code: DSA03

- Semester: 5
- Mobile Number: 9876543210

5. Output

Register Number: VALID

Institutional Email: VALID

Course Code: VALID

Semester: VALID

Mobile Number: VALID

Registration Status: SUCCESSFUL

Result — Step-by-step explanation

Step 1: Read the registration details.

The program receives the following sample values:

- Register number: 192472193
- Institutional email: charishma@simats.ac.in
- Course code: DSA03
- Semester: 5
- Mobile number: 9876543210

These values are checked against the validation rules defined in the program.

Step 2: Validate the register number.

The program uses:

re.fullmatch(r"\d{9}", register_no)

Here, \d matches a digit, and {9} requires exactly nine digits.

The register number 192472193 contains exactly nine digits, so it is VALID.

If the register number were 12345, it would be invalid because it contains only five digits.

Step 3: Validate the institutional email.

The program uses:

re.fullmatch(r"[\w.-]+@simats\.ac\.in", email)

The pattern checks for a username, followed by @simats.ac.in. The \. symbols match literal dots.

The email charishma@simats.ac.in matches the required format, so it is VALID.

An email such as charishma@gmail.com is invalid because it does not use the required institutional domain.

Step 4: Validate the course code.

The program uses:

re.fullmatch(r"[A-Za-z]{2,5}\d{2,4}", course_code)

The pattern requires two to five letters followed by two to four digits.

The course code DSA03 contains three letters (DSA) and two digits (03), so it is VALID.

The code D03 is invalid because it contains only one letter.

Step 5: Validate the semester.

The program checks:

semester.isdigit() and 1 <= int(semester) <= 8

The semester value 5 contains digits and falls within the allowed range of 1 to 8. Therefore, it is VALID.

A semester value of 9 would be invalid because it exceeds the permitted range.

Step 6: Validate the mobile number.

The program uses:

re.fullmatch(r"[6-9]\d{9}", mobile)

The pattern requires exactly ten digits, with the first digit between 6 and 9.

The mobile number 9876543210 satisfies both conditions, so it is VALID.

The number 5876543210 is invalid because it starts with 5.

Step 7: Display the validation status.

The program stores each validation result as True or False in the validations dictionary. It then prints VALID or INVALID for every field.

Finally, all(validations.values()) checks whether every field passed validation. Since all five sample values are valid, the program displays:

Registration Status: SUCCESSFUL

If even one field fails validation, the registration status becomes UNSUCCESSFUL.

Overall result: The university registration validation system successfully checks the register number, institutional email, course code, semester, and mobile number using regular expressions and a numeric range check. All five sample inputs satisfy the required rules, so the registration is marked SUCCESSFUL.